

Git & GitHub Workshop

Easy Companion Handbook · 2-Day Workshop

Day 1 + Day 2

Edition: v2025.1

§1 — A Short History: Why Linus Torvalds Made Git

In **April 2005**, **Linus Torvalds** — the creator of the Linux kernel — built Git in under two weeks after BitKeeper stopped offering its tool for free. He needed something that could handle **millions of lines of C code** from thousands of people around the world — fast, with no single computer in charge, and easy branching.

- ◆ **Speed** — things like commit, diff, branch, and log all happen on your own computer, no internet needed.
- ◆ **Safety** — every piece of data gets a SHA-1 code attached, so silent, hidden data damage is basically impossible.
- ◆ **Many branches at once** — thousands of branches get combined every day inside the Linux project.

§2 — What Is Git, Really?

Think of a C program as a set of `.c / .h` files that get compiled into one program. Git treats every commit as a **point in a big tree of snapshots**, where each point remembers the point that came right before it. Git is not just a backup tool — it is a **system that tracks every small, line-by-line change** in every file you have ever saved.

★ **NOTE:** *Like in C: A commit hash works like a pointer to a node in a linked list. `git log` walks backward through time using these pointers — just like moving through a linked list where every node points back to the one before it.*

§3 — Your Local Setup: Three Simple Stages

Every file moves through three stages before it becomes a permanent commit. It is a bit like C compiling: source file → object file → finished program.

① Working Directory	② Staging Index	③ Local Repository
Your plain files on disk. Changes here are <i>untracked</i> or <i>modified</i> . Git can see the changes but hasn't saved them yet.	A holding area before you commit. It lists what will go into the next snapshot — like a C header listing what to compile. <code>git add</code> puts files here.	The hidden <code>.git/</code> folder. It stores every save as a permanent, unchangeable record. <code>git commit</code> adds a new one here.

► How Git Works on Your Computer



Workshop Roadmap — What This Handbook Covers

DAY 1 — Working Locally	DAY 2 — Collaboration & Conflicts
<code>git init</code> , setting your name/email, what's inside a commit, status/diff/log/add/commit/revert — pages 1-2 of this handbook.	SSH setup, clone/fetch/merge/pull, fixing merge conflicts, and the Fork → PR way of contributing — pages 3-4.

★ **NOTE:** *Before you start: you should already be comfortable compiling a multi-file C project by hand — Git's way of thinking builds on the same ideas you already know: source files, headers, and the compile-link steps.*

§4 — Starting a Repository, Commits & Your Identity

► git init — Starting a New Repository

Running `git init` inside a project folder creates a hidden `.git/` folder — this is where everything about your repository lives. Key parts inside it:

- ◆ `.git/objects/` — stores your files, folders, and commits (compressed)
- ◆ `.git/refs/heads/` — one small file per branch, holding its latest commit ID
- ◆ `.git/HEAD` — a one-line file that points to your current branch

```
$ git init my-project
Initialized empty Git repository in /home/user/my-project/.git/

$ ls -la my-project/.git/
HEAD  config  description  hooks/  info/  objects/  refs/
```

► git config --global — Setting Up Your Name and Email

Every commit saves your name and email forever, as part of its ID. Setting this up once is like writing `#define AUTHOR` in a shared header file — it applies to every repository on this computer:

```
$ git config --global user.name "Gaurav Kathayat"
$ git config --global user.email "gaurav@example.com"
$ git config --list
user.name=Gaurav Kathayat      user.email=gaurav@example.com
```

► A Closer Look: What's Inside a Commit

Field	What It Stores	Example Value
tree	Points to the snapshot of all your files	a3f9c1e8...
parent	The ID of the commit right before this one — a link back in time	8d4e7b20...
author	Name, email, and time of whoever made the change	Gaurav <g@e.com> 1720000000
committer	Who actually saved the commit — can differ from author after git rebase	Same or different
message	A short note about the change (your commit -m text)	"Fix null-check in utils.c"

► Checking Things: git status | git diff | git log

git status — what changed:

```
$ git status
On branch main
Changes not staged:
  modified: utils.c
Untracked files:
  newfile.c
```

git diff — the exact lines:

```
$ git diff utils.c
- if (idx < 0)
+ if (idx <= 0)
  return -1;
```

git log --oneline — history:

```
$ git log --oneline -3
a3f9c1e Fix null-check
8d4e7b2 Add validation
3c1a0f8 Initial scaffold
```

`git diff` shows exactly which lines changed — added (+) or removed (-) — before you commit.

► Saving Changes: git add | git commit

git add — move files into the staging area:

```
$ git add utils.c      # single file
$ git add .            # ALL changes
# utils.c is now staged
```

git commit -m — save the staged changes:

```
$ git commit -m "Fix: null check"
[main a3f9c1e] Fix: null check
1 file changed, 3 insertions(+)
```

► git revert <hash> — Undo Without Erasing History

`git revert` makes a brand-new commit that *undoes* an older one — without erasing anything. Unlike `git reset --hard`, the old commit is still kept safely in history, so it's safe to use on branches other people share:

```
$ git revert a3f9c1e
[main 7c2d0f1] Revert "Fix: null check"
1 file changed, 3 deletions(-)

# Commit a3f9c1e is PRESERVED in history.
# New commit 7c2d0f1 applies the exact inverse patch.
```

★ **NOTE:** Like in C: `git revert` is like flipping a patch with `NOT` — the change history stays whole. `git reset --hard` deletes history — risky on shared branches, like cutting a linked list in the middle.

\$5 — Working With a Remote: GitHub

► What Is GitHub?

GitHub is a Git server in the cloud. It keeps a copy of your `.git/` folder on a computer anyone can reach online, plus a website for reviewing code, running automated tests, and tracking issues. Many people can push and pull from the same project at the same time.

► Setting Up a Secure Connection (SSH)

HTTPS asks for your password every time you push. **SSH keys** fix this with a one-time setup: your **public key** goes to GitHub, and your **private key** stays on your computer. Think of it like this: public key = `.h` file (safe to share); private key = `.c` file (keep it secret).

```
[1] Generate key pair:  $ ssh-keygen -t ed25519 -C "gaurav@example.com"
    Generating public/private ed25519 key pair... (Enter = default path)
[2] Load into SSH agent: $ eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519
    Identity added: id_ed25519
[3] Copy PUBLIC key:    $ cat ~/.ssh/id_ed25519.pub
    ssh-ed25519 AAAA...xyz gaurav@example.com ← paste into GitHub Settings
[4] Check the connection: $ ssh -T git@github.com
    Hi GauravK! You have successfully authenticated.
```

► Connecting Your Local Repo to GitHub

```
$ git remote add origin git@github.com:GauravK/my-project.git
$ git remote -v
origin  git@github.com:GauravK/my-project.git (fetch)
origin  git@github.com:GauravK/my-project.git (push)
$ git push -u origin main
# first push + link the branch
Branch "main" set up to track remote branch "main" from "origin".
```

► git clone — Copying a Whole Project From GitHub

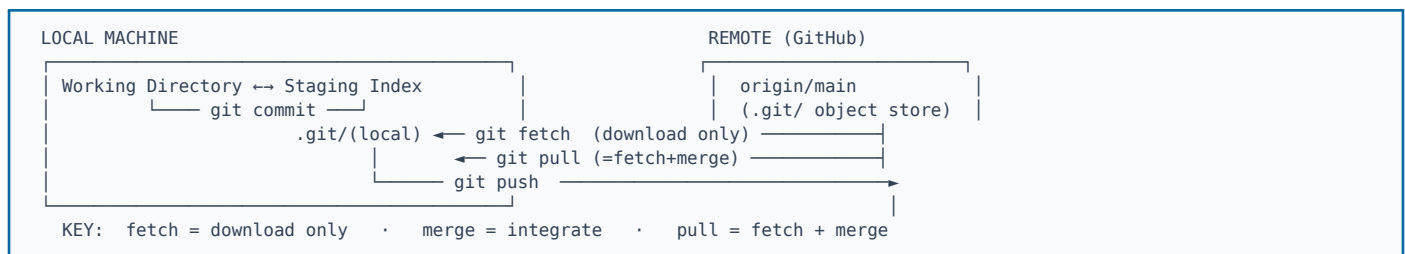
git clone <URL> is not the same as **git init**. `git init` creates an empty repository, but `git clone` downloads the *whole project history*, unpacks it, and automatically sets up `origin` for you — ready to push and pull right away:

```
$ git clone git@github.com:GauravK/my-project.git
Cloning into "my-project"...
remote: Enumerating objects: 47, done.
Receiving objects: 100% (47/47), 23.14 KiB | 1.2 MiB/s, done.
Resolving deltas: 100% (12/12), done.
```

► git fetch, git merge, git pull — What Each One Does

Command	What It Does	Changes Your Files?
<code>git fetch origin</code>	Downloads new commits from GitHub and updates <code>origin/main</code> . It only reads — your working files stay untouched.	NO
<code>git merge origin/main</code>	Combines the downloaded changes into your current branch, either by fast-forwarding or making a merge commit.	YES
<code>git pull</code>	Runs <code>git fetch</code> and then <code>git merge</code> , one after the other. In short: <code>git pull</code> = <code>git fetch</code> + <code>git merge</code> .	YES

► How Your Computer Talks to GitHub



§6 — Fixing Merge Conflicts & Contributing on GitHub

► What a Merge Conflict Looks Like (C Example)

Example: Alice (on feature-alice) and Bob (on feature-bob) both change the same bounds-check line in `utils.c`. When Alice runs `git merge feature-bob`, Git can't decide which version to keep — so it marks the conflict right inside the file:

```
/* utils.c — RAW CONFLICT STATE after: $ git merge feature-bob */

int check_index(int idx, int size) {

<<<<<<< HEAD          /* ← ALICE's branch content (current HEAD) */
    if (idx < 0 || idx >= size) return -1; /* Alice: strict upper bound */
=====              /* ← DIVIDER — below is BOB's version only */
    if (idx < 0 || idx > size-1) return -2; /* Bob: adjusted bound */
>>>>>>> feature-bob    /* ← End of BOB's incoming changes */

    return data[idx]; /* ← Shared code below the conflict is unchanged */
}
```

What the three conflict markers mean:

- ◆ <<<<<<< HEAD — start of YOUR version (what the current branch has at HEAD).
- ◆ ===== — divider; everything below this line came from the incoming branch.
- ◆ >>>>>>> feature-bob — end of the INCOMING branch's changes.

How to fix it — 4 steps:

- [1] Open `utils.c` in your editor. Decide which version of the code is correct (e.g., Alice's `>= size` for a strict upper bound), then **delete all three marker lines completely**.
- [2] Save the file — it should now be normal, working C code with no markers left.
- [3] Stage the fixed file: `$ git add utils.c`
- [4] Finish the merge: `$ git commit` (Git will fill in a default message).

► Contributing to Open Source: Fork → Branch → Pull Request

Forking gives you your own copy of a project on GitHub. You never push directly to someone else's repository. Instead: fork it → clone it → make changes → push to your fork → open a **Pull Request (PR)** asking the original project to review and add your changes.

- [1] **Fork on GitHub** — click "Fork" on the project's page. GitHub makes you your own copy at `YourName/project-name`.
- [2] **Clone your fork locally:** `$ git clone git@github.com:YourName/project-name.git`
- [3] **Create a new branch for your change:** `$ git checkout -b feature-fix` or `$ git switch -c feature-fix` (Git 2.23+)
- [4] **Push your branch to your fork:** `$ git push origin feature-fix`
- [5] **Open a Pull Request on GitHub** — go to the original project, click "Compare & pull request", choose `upstream:main` as the target, write a clear description, and submit. The maintainers will review it, leave comments, ask for changes, or approve and merge.

► Quick Command Reference

Command	Scope	Tip
<code>git init</code>	Local	Run this once, in your project's main folder — not inside another git repo.
<code>git add</code>	Local	Use <code>git add -p</code> to stage your changes piece by piece.
<code>git commit</code>	Local	Write short, clear messages: "Fix bug" instead of "Fixed bug".
<code>git push</code>	Network	Use <code>-u</code> the first time to link your branch; after that, just <code>git push</code> .
<code>git clone</code>	Network	Downloads the whole project and sets up origin for you automatically.
<code>git fetch</code>	Network	Safe to run anytime — only downloads. Check origin/main with <code>git log</code> first.
<code>git merge</code>	Local	Use <code>--no-ff</code> to keep a clear record of feature branches in your history.
<code>git pull</code>	Network+Local	Same as <code>git fetch</code> + <code>git merge</code> . Add <code>--rebase</code> for a cleaner, straight history.
<code>git checkout</code>	Local	Switches branches or restores files. <code>git switch</code> is the newer, simpler option.